RAMDEOBABA UNIVERSITY

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

B. TECH PROJECT REPORT
(Major/Honor/Minor/Idea Lab/External)

ON

I-JEPA-AD: DECODER-FREE LATENT ANOMALY DETECTION WITH COARSE-TO-FINE ADAPTIVE MASKING

Submitted by

Aryan Jawanjal	(Roll No. A-35)
Akshat Tiwari	(Roll No. A-11)
Arnav Milmile	(Roll No. A-30)
Aashutosh Dubey	(Roll No. A-02)

Under the Guidance of

Dr. Sunita G. Rawat

Academic Year: 2026–2027

School of Computer Science and Engineering
Ramdeobaba University, Nagpur

STUDENT DECLARATION

We hereby declare that the project report entitled **"I-JEPA-AD: Decoder-Free Latent Anomaly Detection with Coarse-to-Fine Adaptive Masking"** is submitted as part of the requirements for the B.Tech programme at Ramdeobaba University.

The work presented in this report is our own work carried out under the guidance of the project supervisor. All external sources, datasets, software tools, figures and technical material used in the project have been appropriately acknowledged and cited.

Aryan Jawanjal
Roll No. A-35

Akshat Tiwari
Roll No. A-11

Arnav Milmile
Roll No. A-30

Aashutosh Dubey
Roll No. A-02

APPROVAL SHEET

This is to certify that the project report entitled **"I-JEPA-AD: Decoder-Free Latent Anomaly Detection with Coarse-to-Fine Adaptive Masking"** has been carried out by the students listed below under the guidance of the undersigned and is submitted in partial fulfilment of the academic requirements of the B.Tech programme.

Aryan Jawanjal, Akshat Tiwari, Arnav Milmile, Aashutosh Dubey

Guide Dr. Sunita G. Rawat	Project In-Charge	Associate Head
(Signature)	(Signature)	(Signature)

Dean, CSE	External Examiner
(Signature)	(Signature)

ACKNOWLEDGEMENT

We express our sincere gratitude to Ramdeobaba University and the School of Computer Science and Engineering for providing the academic environment and facilities required to complete this project.

We are grateful to our project guide, Dr. Sunita G. Rawat, for continuous guidance, technical suggestions and constructive feedback throughout the project. We also thank the Project In-Charge, Associate Head and Dean, School of Computer Science and Engineering, for their support.

We acknowledge the authors of I-JEPA and of the MVTec AD dataset, whose public models and data made this study possible. Finally, we thank our classmates, friends and family members who contributed directly or indirectly to the successful completion of this work.

ABSTRACT

Unsupervised visual anomaly detection for industrial inspection is dominated by pixel-reconstruction models, which need heavy decoders, and by nearest-neighbour feature banks, whose memory grows with the training set. This project studies a third option, I-JEPA-AD: a decoder-free detector that predicts masked image regions entirely in the latent space of a frozen I-JEPA ViT-g/16 encoder. A small Transformer predictor is trained only on cached latents of defect-free images, and anomalies are scored by the cosine discrepancy between predicted and actual latents. A coarse-to-fine masking scheme evaluates sixteen coarse blocks first and runs single-patch fine masks only where a gate flags a suspicious region.

On a three-category pilot of MVTec AD (bottle, capsule, hazelnut) the original pipeline reaches a mean pixel AUROC of 0.840 with about 16 masks per image, a 92 percent reduction from 196 dense masks, and an end-to-end latency of 134 ms against 350 ms for dense masking on an 8 GB RTX 4060. However, the fine stage almost never fires with the default gate, capsule stays near chance, and a nearest-neighbour baseline on the same latents is stronger. We then choose settings on synthetic defects pasted onto nominal validation images, never on the test split. Using mid-level encoder tokens (layer 32 of 40) and normalising scores per token position raises the mean top-3 image AUROC from 0.787 to 0.920, the mean pixel AUROC from 0.840 to 0.962 and the mean AUPRO from 0.480 to 0.813 at the same 16 masks per image, with paired-bootstrap improvements over the original pipeline in all three categories. The report also documents negative results and the limitations of the pilot.

Keywords: Anomaly Detection, I-JEPA, Latent Prediction, Masked Modelling, Industrial Inspection, MVTec AD

Contents

Student Declaration i

Approval Sheet ii

Acknowledgement iii

Abstract iv

List of Abbreviations vii

1 Introduction 2

1.1 Background . . . 2
1.2 Problem Statement . . . 2
1.3 Objectives . . . 3
1.4 Scope of the Project . . . 3
1.5 Organization of the Report . . . 3

2 Literature Review and Research Gap 5

2.1 Introduction . . . 5
2.2 Existing Approaches . . . 5
2.3 Comparison of Existing Approaches . . . 6
2.4 Limitations of Existing Work . . . 6
2.5 Research Gap . . . 6

3 Proposed System and Methodology 8

3.1 Proposed System . . . 8
3.2 Data Acquisition . . . 8
3.3 Data Preprocessing and Latent Cache . . . 9
3.4 Analytical Method . . . 9
3.4.1 Latent predictor . . . 9
3.4.2 Coarse-to-fine inference . . . 10
3.5 Validation-Based Selection Protocol . . . 10
3.6 Evaluation Metrics . . . 11

4 System Design and Implementation **13**
4.1 System Architecture . . . 13
4.2 Hardware Components . . . 13
4.3 Software Components . . . 13
4.4 Latent Cache Design . . . 14
4.5 User Interface . . . 14
4.6 Implementation Workflow . . . 14

5 Results and Discussion **17**
5.1 Experimental Setup . . . 17
5.2 Results of the Original Pipeline . . . 17
5.2.1 Detection and localisation . . . 17
5.2.2 Masking policy: accuracy versus cost . . . 18
5.2.3 Latency and memory . . . 19
5.2.4 Validation-only gate calibration . . . 20
5.3 Results of the Validation-Selected Improvements . . . 21
5.3.1 Selecting on synthetic defects . . . 21
5.3.2 Effect on the test split . . . 22
5.3.3 A negative result: re-selection for layer 32 . . . 24
5.3.4 Pretrained predictor and context leakage . . . 24
5.4 Discussion . . . 24
5.5 Limitations . . . 25

6 Conclusion and Future Scope **27**
6.1 Conclusion . . . 27
6.2 Future Scope . . . 27

References **29**

A Publications and Intellectual Property **31**

B Student Contribution Statement **32**

C Project Outputs **33**

List of Figures

3.1 I-JEPA-AD overview. (a) Latents of nominal images are cached once and used to train the predictor. (b) At test time a coarse pass is followed by a gated fine pass, and the two score maps are fused. 8

5.1 Image AUROC (top-5 mean score) of the baseline and augmented predictors versus the nearest-neighbour reference on the test split. 18
5.2 Raw cosine-discrepancy maps for one pronounced defect per category (top to bottom: bottle, capsule, hazelnut). Columns: image, ground truth, coarse map, gated fine map, fused adaptive map and the dense reference. 19
5.3 End-to-end latency per image split into encoder and scoring time. . . . 20
5.4 Synthetic validation defects (CutPaste and Perlin-noise blends) on nominal validation images; the defect outline is shown in red. 22
5.5 Layer-32 predictor with the selected Mahalanobis scoring on the test split, compared with the other rows of Table 5.6. 23

List of Tables

2.1 Comparison of unsupervised anomaly detection approaches. 6

3.1 Pilot split sizes (images). 9

5.1 Test results on three MVTec AD categories for the original pipeline. "Default gate" uses $\tau = 0.5$; "calibrated gate" uses the 0.99 token quantile of nominal validation scores; the reference is a nearest-neighbour score on the same latents. 18

5.2 Pixel AUROC and AUPRO per category (B/C/H = bottle/capsule/hazelnut) and mean predictor passes per image for four masking policies (baseline predictors, test split). 19

5.3 Latency and memory on an RTX 4060 (8 GB), FP32 encoder. End-to-end time is the mean $\pm$ standard deviation over 40 images; the last column is predictor-only scoring on cached latents per category (bottle/capsule/hazelnut). 20

5.4 Gate calibration on nominal validation images. For each category: τ / fine windows per nominal image / % of nominal images triggering a fine pass. 21

5.5 Paired-bootstrap change in image AUROC versus the original pipeline (default gate, max score), 95 % interval. Asterisk: interval excludes zero. 21

5.6 Test results after the validation-selected changes. Img AUROC is given for the max score and for the mean of the top-3 token scores. (a) Layer-32 settings re-selected on synthetic data (Mahalanobis residual score, max score); (b) nearest-neighbour reference on layer-32 tokens (full memory bank), context only. 23

5.7 Paired-bootstrap change in image AUROC on the test split, 95 % interval. Asterisk: interval excludes zero. The first four rows use configurations fixed before their test evaluation; the last three evaluate the layer-32 re-selection. 23

A.1 Publications and Intellectual Property 31

B.1 Student Contribution Statement 32

LIST OF ABBREVIATIONS

AUROC	Area Under the Receiver Operating Characteristic curve
AUPRO	Area Under the Per-Region-Overlap curve
GPU	Graphics Processing Unit
I-JEPA	Image-based Joint-Embedding Predictive Architecture
kNN	k-Nearest Neighbour (here, nearest-neighbour feature bank)
MAE	Masked Autoencoder
MVTec AD	MVTec Anomaly Detection dataset
RBU	Ramdeobaba University
SHA	Secure Hash Algorithm
UI	User Interface
ViT	Vision Transformer

CHAPTER

1

1. INTRODUCTION

1.1 Background

Automated detection and localisation of surface defects is central to industrial quality control. Defective samples are rare, and the types of defect that may appear are open-ended, so the standard setting is *unsupervised*: a model is trained on defect-free (nominal) images only and must flag anything that deviates from them at test time [3, 4]. The MVTec AD dataset is the usual benchmark for this task.

Two families of methods dominate. Reconstruction-based approaches (autoencoders, generative networks, masked autoencoders and inpainting networks) assume that anomalies cannot be reconstructed well [2, 8, 7]. They pay for a pixel decoder, spend capacity on irrelevant high-frequency texture, and can generalise so well that they reconstruct the defect. Embedding-based approaches such as PaDiM and PatchCore compare test patch features with those of nominal images [6, 5]. They are accurate, but PatchCore stores a memory bank whose size scales with the number of training patches.

The Joint-Embedding Predictive Architecture (I-JEPA) offers a different route: a model learns to predict the *representations* of masked image blocks from visible context, without ever generating pixels [1, 12]. This suggests an anomaly detector with no decoder, in which a region is anomalous when its latent representation cannot be predicted from its surroundings.

1.2 Problem Statement

Given only defect-free images of an object category, the task is to decide whether a new image contains a defect (image-level detection) and to indicate where it is (pixel-level localisation). Using I-JEPA for this raises two practical issues. First, evaluating the predictor over every patch with a dense sliding window is redundant, since most of an industrial image is nominal. Second, the frozen ViT-g/16 backbone has about one billion parameters, so training and inference must fit a consumer GPU with 8 GB of memory.

1.3 Objectives

The objectives of the project are:

- To build a decoder-free pipeline that caches frozen I-JEPA ViT-g/16 latents offline, trains a lightweight Transformer predictor on nominal latents only, and scores anomalies by cosine discrepancy.
- To design a coarse-to-fine inference scheme that spends predictor passes only where a gate flags a suspicious region.
- To define a validation-only protocol for choosing settings, so that no choice is tuned on the test labels.
- To measure accuracy, number of predictor passes, latency and GPU memory against fixed-mask, dense-mask and nearest-neighbour references.
- To improve the detector through choices made on synthetic validation data, and to report negative results honestly.

1.4 Scope of the Project

The study is a pilot on three of the fifteen MVTec AD categories (bottle, capsule and hazelnut) at the native 224×224 input size, with a frozen encoder and a single consumer GPU. The remaining twelve categories, MVTec AD 2, a faithful PatchCore and MAE baseline, multi-seed runs and higher input resolution are outside the scope of the work reported here and are listed as future scope. The results are therefore pilot evidence and not a benchmark claim.

1.5 Organization of the Report

Chapter 2 reviews related work and states the research gap. Chapter 3 describes the proposed system and the methodology, including the validation-based selection protocol. Chapter 4 covers the system design and implementation. Chapter 5 presents the experimental setup, results, discussion and limitations. Chapter 6 concludes and lists future scope. Appendices give the publication status, the contribution of each student and the project outputs.

CHAPTER

2

2. LITERATURE REVIEW AND RESEARCH GAP

2.1 Introduction

This chapter reviews the families of unsupervised anomaly detection methods that are relevant to the project and identifies the gap that motivates a latent-prediction detector.

2.2 Existing Approaches

Reconstruction-based detection. Autoencoder, generative and masked-autoencoder (MAE) detectors reconstruct nominal images and score by reconstruction error [2, 7]. Reconstruction by inpainting reduces the tendency to reconstruct defects by hiding regions before reconstruction [8]. All of these need a decoder that operates in pixel space.

Embedding-based detection. PaDiM models per-position Gaussian statistics of pretrained features [6]. PatchCore stores a coreset of nominal patch features and scores a test patch by its nearest-neighbour distance [5]. EfficientAD targets millisecond-level latency with a lightweight student–teacher design [10]. Libraries such as Anomalib standardise the evaluation of these methods [11].

Joint-embedding prediction. I-JEPA learns by predicting target-block representations from a context block in latent space, which avoids pixel-level detail and hand-crafted augmentations [1]. Its use for *anomaly localisation*, and in particular how to spend predictor evaluations economically, has received little study.

Synthetic defects. Pasting or blending synthetic defects onto normal images is an established way to obtain labelled anomalies without real defect data; CutPaste and DRAEM are well-known examples [9, 7]. In this project such defects are used only to choose settings, never to train the detector.

2.3 Comparison of Existing Approaches

Table 2.1: Comparison of unsupervised anomaly detection approaches.

Approach	Principle	Needs pixel decoder	Main limitation
Reconstruction (AE, MAE, inpainting)	Reconstruct input; score the error	Yes	Defects may be reconstructed; capacity spent on texture
PaDiM	Per-position Gaussian of pre-trained features	No	Fixed statistics may be too coarse for small defects
PatchCore	Nearest neighbour to a bank of nominal patches	No	Memory grows with training patches
I-JEPA-AD (this work)	Predict masked latents; score the discrepancy	No	Many predictor passes; weaker than a bank in this pilot

2.4 Limitations of Existing Work

Decoder-based methods carry the cost and the failure modes of pixel reconstruction. Feature-bank methods are strong but trade memory for accuracy, and their memory footprint grows with the amount of nominal data. Methods based on latent prediction have not been studied systematically for localisation, and there is no established way to decide where the (expensive) predictor should be evaluated.

2.5 Research Gap

To the authors' knowledge, a frozen I-JEPA predictor has not been studied as a decoder-free anomaly detector with an adaptive masking policy and a protocol that selects its settings without touching the test labels. This project addresses that gap with a measured comparison of masking policies, a validation-only selection protocol and an explicit account of where the approach fails.

CHAPTER

3

3. PROPOSED SYSTEM AND METHODOLOGY

3.1 Proposed System

Figure 3.1 gives an overview. The I-JEPA encoder is frozen throughout and only the predictor is trained. Latents of nominal images are computed once and cached; the predictor is trained on the cache. At test time a coarse pass is followed by a gated fine pass and the two score maps are fused into a heat map and an image score.

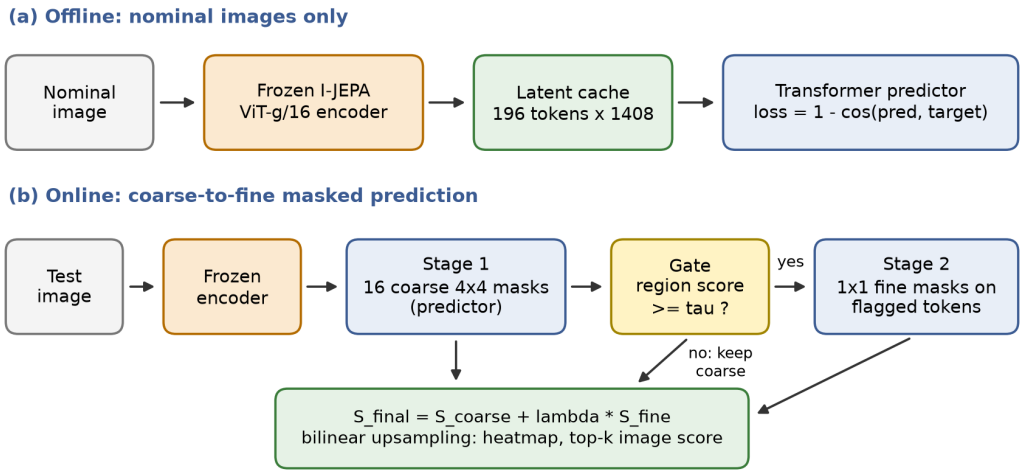


Figure 3.1: I-JEPA-AD overview. (a) Latents of nominal images are cached once and used to train the predictor. (b) At test time a coarse pass is followed by a gated fine pass, and the two score maps are fused.

3.2 Data Acquisition

The pilot uses the categories *bottle*, *capsule* and *hazelnut* of MVTec AD [3]. The official training set (nominal images only) is split by image-content hash into 90 % predictor-training images and 10 % validation images (seed 0). The validation images are used only for early stopping, calibration and selection; the official test set is touched only for final metrics. Table 3.1 lists the sizes. The split was verified to have no overlap between training, validation and test images.

Table 3.1: Pilot split sizes (images).

Category	Train	Val	Test	Test good	Test defect
Bottle	188	21	83	20	63
Capsule	197	22	132	23	109
Hazelnut	352	39	110	40	70

3.3 Data Preprocessing and Latent Cache

An image is resized to 224×224 and normalised with ImageNet statistics. The official I-JEPA ViT-g/16 (ImageNet-22K) target encoder, with about 1.01×10^9 parameters, maps it to a 14×14 grid of 196 patch tokens $z_i \in \mathbb{R}^{1408}$. Since the encoder is frozen, every image is encoded once and the latents are stored in a cache whose identity includes the checkpoint hash, the preprocessing specification and the SHA-256 of the image content. This decouples predictor training from the 16.5 GB checkpoint and keeps training within an 8 GB budget.

3.4 Analytical Method

3.4.1 Latent predictor

The predictor receives all 196 latents together with a binary target mask $m \in \{0,1\}^{196}$. Masked tokens are replaced by a learned mask token, a learned positional embedding is added, and a Transformer encoder (hidden size 512, 4 layers, 8 heads, about 14 M parameters) outputs a prediction $\hat{z}_i \in \mathbb{R}^{1408}$ for every token. The model is trained on nominal images only with the cosine loss

$$\mathcal{L} = \frac{1}{|M|} \sum_{i \in M} \big(1 - \cos(\hat{z}_i, z_i)\big), \tag{3.1}$$

where M is the set of masked tokens and the target z_i is the cached latent. The loss is evaluated on masked tokens only, and the masked input is replaced by the mask token before any computation, so target latents cannot reach the predictor.

Inference-aligned training masks. Inference scores either one 4×4 coarse block or single tokens, so training masks follow the same geometry: with probability 0.5 a row hides one of the grid-aligned 4×4 windows, otherwise it hides up to eight mutually non-adjacent single tokens.

3.4.2 Coarse-to-fine inference

The anomaly score of a token is the cosine discrepancy $s_i = 1 - \cos(\hat{z}_i, z_i) \in [0, 2]$.

Stage 1 (coarse). The grid is partitioned into 16 non-overlapping 4×4 windows. Each window is masked in turn and the predictor yields s_i for its tokens, giving the coarse map S_{coarse} with 16 predictor passes. The region score r_i of a token is the mean coarse score of its window.

Stage 2 (adaptive fine). Every token whose region score satisfies $r_i \geq \tau$ receives its own 1×1 mask and an extra predictor pass, giving S_{fine} (zero elsewhere). If the gate never fires, the method reduces to fixed coarse masking; if it fires everywhere, it becomes dense masking with 196 single-token masks.

Fusion and image score. The final map is

$$S_{\text{final}} = S_{\text{coarse}} + \lambda \, S_{\text{fine}}, \tag{3.2}$$

bilinearly upsampled to image resolution for pixel-level evaluation. The image score is the maximum of S_{final} or the mean of its k largest values (top-k).

Cost model. Fixed masking costs 16 predictor passes, dense masking 196 and adaptive masking $16 + n_f$, where n_f is the number of tokens passing the gate. The encoder runs once per image in all cases.

3.5 Validation-Based Selection Protocol

The nominal validation images contain no defects, so nothing can be chosen for *detection* quality from them alone. The project therefore builds a labelled synthetic validation set and uses it, with a fixed protocol, to choose settings without reading the test split.

Synthetic-defect set. For every nominal validation image four defects are generated with a fixed seed, alternating CutPaste scars and DRAEM-style Perlin-noise texture blends. Defects stay on the object and cover 0.3–4 % of the image; textures come from nominal training images. This gives 84, 88 and 156 synthetic images for bottle, capsule and hazelnut, each with an exact defect mask.

Selection rules, declared before any result was seen. Statistics that are fitted (per-position z-score, Mahalanobis covariance, gate quantiles) are cross-fitted over three folds of the clean validation images, so no image is used both for fitting and for evaluation. Settings are chosen greedily in a fixed order (score family, smoothing σ, image-score k, fusion λ, gate τ) with one criterion, the mean over categories of the average of image AUROC, pixel AUROC and AUPRO, and one global choice for all categories.

Options within 0.005 of the best are treated as tied and the earlier, simpler option wins.

Encoder layer. Tokens from blocks 8, 12, …, 40 of the frozen encoder (with a parameter-free per-token layer norm) were scored with a nearest-neighbour reference on the synthetic set, and the same rules chose the layer. Predictors were then trained on those tokens with exactly the baseline recipe, so only the input tokens differ.

3.6 Evaluation Metrics

Image AUROC measures detection. Pixel AUROC and AUPRO (false-positive-rate limit 0.3, 200 thresholds) measure localisation. The mean number of predictor passes (masks) per image and the latency measure cost. Differences in image AUROC are reported with paired-bootstrap 95 % intervals (1,000 resamples; good and anomalous images resampled separately; the same resamples for every comparison). An interval that contains zero is reported as inconclusive. The test split is scored once per reported row and is never used to select a setting.

CHAPTER

4

4. SYSTEM DESIGN AND IMPLEMENTATION

4.1 System Architecture

The implementation is a Python package, `ijepa_ad`, whose modules interact only through public interfaces: data loading and splitting, the frozen encoder wrapper, the latent cache, the predictor and its training loop, the masking and inference module, the scoring variants and the evaluation metrics. A separate Streamlit application provides the interactive front end. Figure 3.1 in Chapter 3 shows the data flow; the implementation follows it exactly.

4.2 Hardware Components

All experiments ran on a single workstation with an NVIDIA GeForce RTX 4060 (8 GB), 15.8 GB of host memory and Windows 11. The frozen ViT-g/16 occupies 3.94 GB as FP32 weights and the official checkpoint is 16.5 GB on disk. The encoder checkpoint is loaded with memory mapping where the full state is not needed, which kept host memory use below 2 GB when the pretrained predictor was examined.

4.3 Software Components

The code is written in Python 3.12 with PyTorch 2.2.2 (CUDA 12.1) and the official I-JEPA source for the encoder. Quality tooling comprises pytest, ruff, black and mypy. Experiments are recorded in Jupyter notebooks, the dashboard uses Streamlit, and this report and the accompanying paper are typeset in LaTeX. Configuration lives in YAML files for data, encoder, predictor, masking, augmentation and frontend settings, so no hyperparameter is hardcoded in the modules.

4.4 Latent Cache Design

Each cache entry stores the 196×1408 token matrix of an image together with metadata: encoder name, checkpoint SHA-256, patch size, embedding dimension, preprocessing specification and the ordered SHA-256 content hashes of the images. A loader rejects any cache whose metadata does not match what the caller expects, and caches are never overwritten. Tokens of an intermediate layer are stored in the same format under a derived checkpoint identity, the hash of the checkpoint, the layer number and the normalisation. A predictor trained on such a cache records that identity, so using it with final-layer latents (or the reverse) raises an error instead of silently mixing feature spaces. Synthetic validation images have their own equally strict cache, which keeps the official test split evaluation-only.

4.5 User Interface

The Streamlit dashboard lets a user select a category and image, switch between the adaptive, dense and fixed policies and the reference methods, and inspect heat maps, overlays with adjustable blending, suspicious regions, mask counts, latency and memory, training loss curves and benchmark summaries. The interface calls a stable function, `run_inference(image, method, category)`, so it does not depend on backend internals.

4.6 Implementation Workflow

1. Split the nominal training images into training and validation sets by content hash and verify no overlap with the test split.
2. Encode every image once with the frozen encoder and write the latent cache.
3. Train one predictor per category on the cache with inference-aligned masks; keep the checkpoint of the best validation epoch together with its provenance (seeds, configuration, image hashes).
4. Score validation and test images with the coarse pass, the gated fine pass and score fusion.
5. Select settings on the synthetic validation set and save the selection as a JSON record before any test evaluation.
6. Score the test split once per row and report metrics with paired-bootstrap intervals.

Testing and reproducibility. The repository contains an automated suite of 376 unit and integration tests covering the data split, caches, predictor training, masking, scoring variants, synthetic-defect generation and the front end. Every code path that touches the encoder asserts that its parameters are frozen. Design decisions, the data flow and the project state are recorded in living documents (`DECISIONS.md`, `FLOW.md`, `TRANSFER.md`), and result tables in this report are taken from saved result files.

CHAPTER

5

5. RESULTS AND DISCUSSION

5.1 Experimental Setup

One predictor is trained per category with AdamW (learning rate 10^{-4}, weight decay 0.05), batch size 16, at most 100 epochs and early stopping with patience 10 on the nominal validation loss, seed 0. The inference defaults are coarse window 4, fine window 1, gate $\tau = 0.5$ and $\lambda = 1$. Latency was measured on the RTX 4060 with an FP32 encoder (30 timed runs after 5 warm-up runs for the encoder; 40 images end to end). As a context row, a PatchCore-style nearest-neighbour score is computed from the *same* frozen final-layer latents: a memory bank of all nominal training patch latents and the L_2 distance of each test patch to its nearest bank patch. This is not the full PatchCore (no mid-level features, no coreset).

5.2 Results of the Original Pipeline

5.2.1 Detection and localisation

Table 5.1 reports the official test split. The adaptive pipeline with the default gate attains a mean pixel AUROC of 0.840 and a mean image AUROC of 0.750 with about 16 masks per image. The nearest-neighbour reference on identical latents is better in every category and metric (mean pixel AUROC 0.956, mean AUPRO 0.747), and capsule image AUROC is near chance (0.522). Adding augmented nominal copies did not help consistently: image AUROC decreased in all three categories (mean $0.750 \rightarrow 0.678$).

Table 5.1: Test results on three MVTec AD categories for the original pipeline. "Default gate" uses $\tau = 0.5$; "calibrated gate" uses the 0.99 token quantile of nominal validation scores; the reference is a nearest-neighbour score on the same latents.

Category	Method	Image AUROC	Image AUROC (top-5)	Pixel AUROC	AUPRO	Masks/img
Bottle	I-JEPA-AD (default gate)	0.925	0.956	0.930	0.766	16.2
	I-JEPA-AD (calibrated gate)	0.951	0.955	0.926	0.732	58.2
	I-JEPA-AD (augmented, default)	0.886	0.966	0.939	0.789	16.0
	PatchCore-style reference	**0.999**	**1.000**	**0.973**	**0.859**	–
Capsule	I-JEPA-AD (default gate)	0.522	0.528	0.684	0.098	16.0
	I-JEPA-AD (calibrated gate)	0.523	0.544	0.700	0.103	23.0
	I-JEPA-AD (augmented, default)	0.398	0.452	0.671	0.098	16.0
	PatchCore-style reference	**0.805**	**0.850**	**0.916**	**0.513**	–
Hazelnut	I-JEPA-AD (default gate)	0.802	0.930	0.907	0.577	16.1
	I-JEPA-AD (calibrated gate)	0.811	0.890	0.923	0.566	22.9
	I-JEPA-AD (augmented, default)	0.750	0.861	0.903	0.600	16.0
	PatchCore-style reference	**0.988**	**0.987**	**0.979**	**0.868**	–
Mean	I-JEPA-AD (default gate)	0.750	0.805	0.840	0.480	16.1
	I-JEPA-AD (calibrated gate)	0.762	0.796	0.850	0.467	34.7
	I-JEPA-AD (augmented, default)	0.678	0.760	0.838	0.496	16.0
	PatchCore-style reference	**0.930**	**0.946**	**0.956**	**0.747**	–

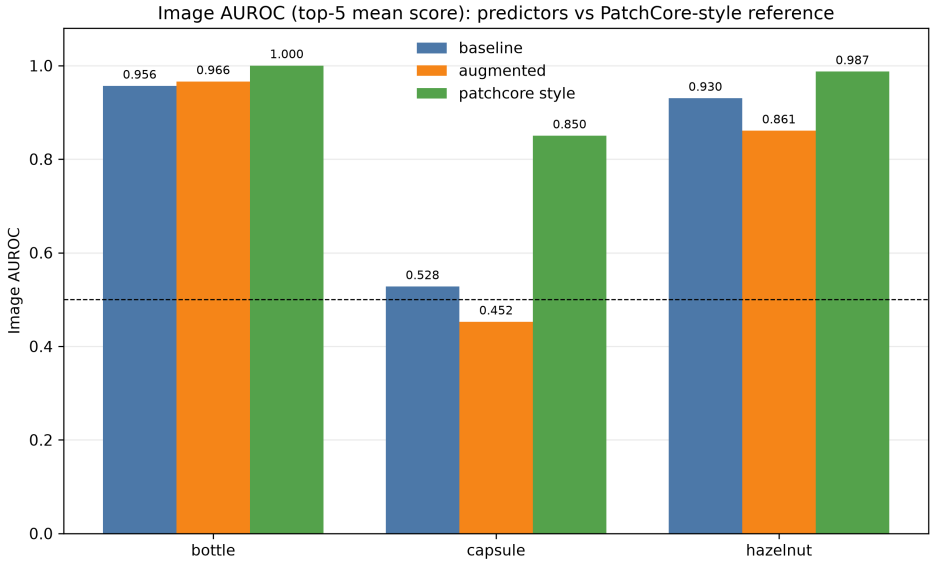


Figure 5.1: Image AUROC (top-5 mean score) of the baseline and augmented predictors versus the nearest-neighbour reference on the test split.

5.2.2 Masking policy: accuracy versus cost

Table 5.2 compares the masking policies with the same baseline predictors. Fixed, default-adaptive and dense masking give nearly identical localisation (mean pixel AUROC 0.840, 0.840 and 0.848) while dense masking needs $196/16 = 12.25\times$ as many predictor passes. Default-adaptive uses only 16.1 masks per image, a 91.8 % reduction from dense. This saving is almost entirely because the default gate opens on only about one test image in a hundred (1.2 %, 0 % and 0.9 % for bottle, capsule, hazelnut), so the

method behaves like fixed coarse masking.

Table 5.2: Pixel AUROC and AUPRO per category (B/C/H = bottle/capsule/hazelnut) and mean predictor passes per image for four masking policies (baseline predictors, test split).

	Pixel AUROC			AUPRO			Masks
Policy	B	C	H	B	C	H	**(mean)**
Fixed (coarse only)	.930	.685	.907	.766	.098	.578	16.0
Adaptive, τ=0.5	.930	.685	.907	.766	.098	.577	16.1
Adaptive, cal. τ	.926	.700	.923	.732	.103	.566	34.7
Dense (196 masks)	.928	.706	.909	.762	.111	.584	196

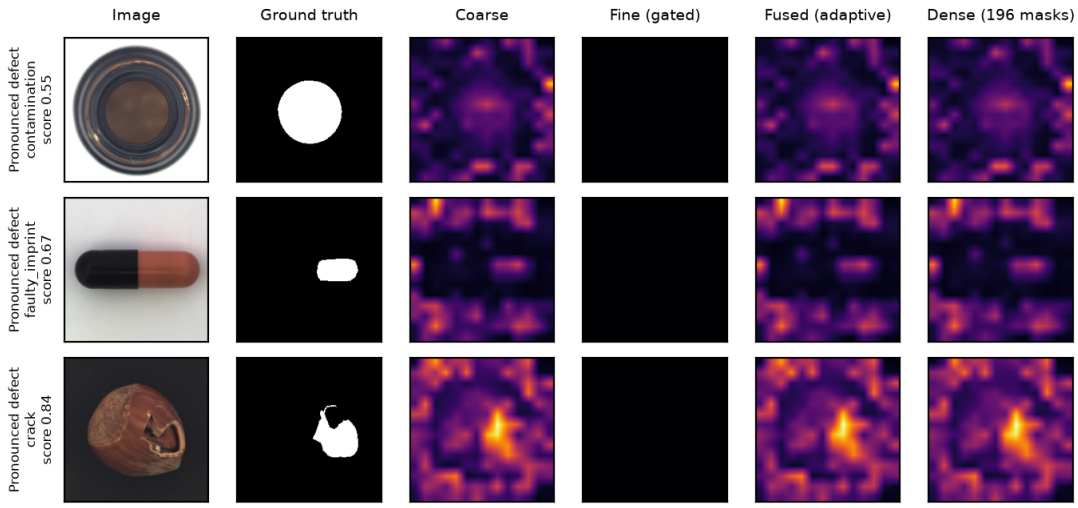


Figure 5.2: Raw cosine-discrepancy maps for one pronounced defect per category (top to bottom: bottle, capsule, hazelnut). Columns: image, ground truth, coarse map, gated fine map, fused adaptive map and the dense reference.

Figure 5.2 shows qualitative maps. For hazelnut the crack is clearly highlighted by both the coarse and the dense maps. For the bottle contamination the discrepancy is spread diffusely and is weaker than the response at the image border, which hurts image-level ranking. For capsule the faulty imprint is only weakly highlighted, consistent with chance-level image AUROC.

5.2.3 Latency and memory

Table 5.3 gives the cost on the 8 GB RTX 4060. The frozen encoder dominates: 3.94 GB of weights and 87.5 ms per image, with a peak memory of 4.08 GB, so the pipeline fits an 8 GB card. Adaptive masking adds 8.9 ms over fixed masking end to end (133.8 against 124.9 ms), while dense masking is 2.6$\times$ slower than adaptive (349.8 ms).

Table 5.3: Latency and memory on an RTX 4060 (8 GB), FP32 encoder. End-to-end time is the mean ± standard deviation over 40 images; the last column is predictor-only scoring on cached latents per category (bottle/capsule/hazelnut).

Policy	End-to-end (ms)	P95 (ms)	Scoring only B/C/H (ms)
Encoder only (batch 1)	87.5 ± 5.4	100.5	–
Fixed	124.9 ± 3.5	130.2	18.1 / 17.5 / 17.5
Adaptive	133.8 ± 4.4	141.3	30.4 / 20.6 / 21.1
Dense	349.8 ± 10.1	367.9	214.4 / 216.7 / 238.1

Peak GPU memory: 4078 MB (all three policies); encoder weights 3939 MB.

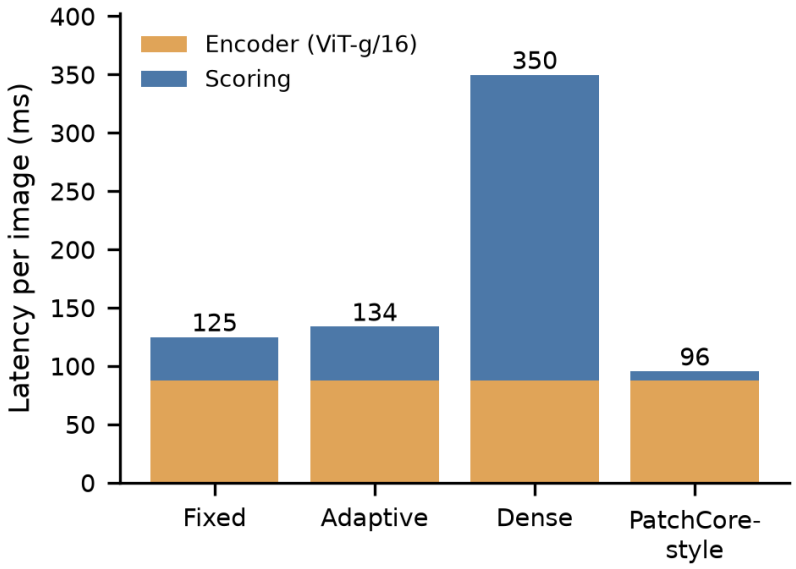


Figure 5.3: End-to-end latency per image split into encoder and scoring time.

5.2.4 Validation-only gate calibration

The default gate is inactive and the max-patch image score lets a single noisy patch decide an image. Calibrating the gate threshold from the q-quantile of nominal validation region scores (Table 5.4) and using a top-5 mean image score were declared before any test result was inspected. The calibrated setting lifts the mean image AUROC from 0.750 to 0.796 and the mean pixel AUROC from 0.840 to 0.850, at 34.7 masks per image instead of 16.1. Table 5.5 shows that the only statistically supported improvement is the top-5 score on hazelnut; everywhere else the interval contains zero.

Table 5.4: Gate calibration on nominal validation images. For each category: τ / fine windows per nominal image / % of nominal images triggering a fine pass.

Gate	Bottle	Capsule	Hazelnut
Default	.500 / 0.0 / 0	.500 / 0.0 / 0	.500 / 0.0 / 0
$q = 0.9$	.098 / 19.8 / 86	.207 / 19.6 / 91	.279 / 19.7 / 92
$q = 0.95$	.123 / 9.9 / 62	.225 / 9.8 / 68	.307 / 9.8 / 74
$q = 0.99$	.158 / 2.3 / 19	.264 / 2.2 / 23	.350 / 2.1 / 23
$q = 0.995$	.184 / 1.1 / 14	.282 / 1.1 / 14	.368 / 1.0 / 13

Table 5.5: Paired-bootstrap change in image AUROC versus the original pipeline (default gate, max score), 95 % interval. Asterisk: interval excludes zero.

Category	Gate only	Top-5 only	Both (primary)
Bottle	+.026 [−.037, +.097]	+.032 [−.005, +.078]	+.030 [−.026, +.097]
Capsule	+.001 [−.114, +.107]	+.006 [−.126, +.153]	+.022 [−.128, +.180]
Hazelnut	+.009 [−.055, +.069]	+.128 [+.068, +.198]*	+.088 [+.006, +.171]*

5.3 Results of the Validation-Selected Improvements

5.3.1 Selecting on synthetic defects

Figure 5.4 shows examples of the synthetic defects. For the final-layer predictor the selection protocol chose a per-token-position z-score of the patch score (each of the 196 positions normalised with the mean and standard deviation of nominal validation scores), $\sigma = 0$, $k = 3$, $\lambda = 1$ and the default gate. After the selection was saved, the proxy's ranking of eight scoring variants agreed with the test split (mean Spearman correlation 0.74 for image AUROC with the max score, 0.86 with top-5, 0.84 for pixel AUROC and 0.94 for AUPRO) but only weakly for capsule's max-score image AUROC (0.42).

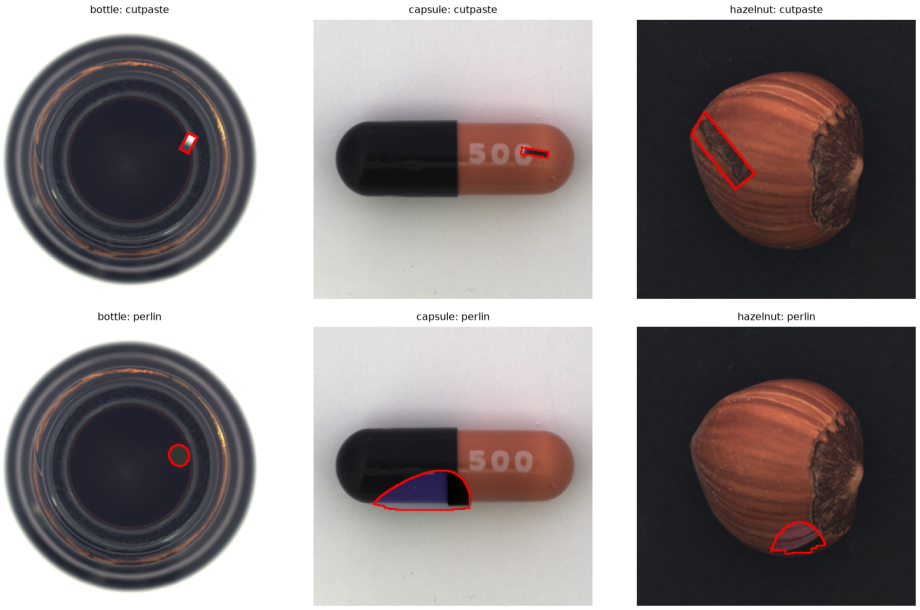


Figure 5.4: Synthetic validation defects (CutPaste and Perlin-noise blends) on nominal validation images; the defect outline is shown in red.

For the encoder layer, mid layers were best on the synthetic criterion (0.987 for layers 24 and 28, 0.986 for layer 32 and 0.969 for the final cached latents). The declared rule selected layer 32; layers 24 and 28 are within the tie margin.

5.3.2 Effect on the test split

Table 5.6 and Table 5.7 report the official test split, with one scoring pass per row. Position z-scoring alone (final layer) raises mean pixel AUROC from 0.840 to 0.941 and mean AUPRO from 0.480 to 0.717. Moving the predictor to layer 32 on top of it raises capsule top-3 image AUROC from 0.672 to 0.767 (+0.095, interval [+0.038, +0.150]) and hazelnut from 0.939 to 0.993 (+0.054, [+0.017, +0.101]); bottle is at the ceiling. The combination improves image AUROC over the original pipeline in all three categories (bottle +0.075, capsule +0.246, hazelnut +0.191; all intervals exclude zero) with the same 16.0–16.6 masks per image and a scorer latency of about 22 ms per image. Mean pixel AUROC reaches 0.962 and mean AUPRO 0.813.

Table 5.6: Test results after the validation-selected changes. Img AUROC is given for the max score and for the mean of the top-3 token scores. (a) Layer-32 settings re-selected on synthetic data (Mahalanobis residual score, max score); (b) nearest-neighbour reference on layer-32 tokens (full memory bank), context only.

Category	Method	Img AUROC (max)	Img AUROC (top-3)	Pixel AUROC	AUPRO	Masks/img
Bottle	Original baseline (final layer, $1-\cos$)	0.925	0.952	0.930	0.766	16.2
	+ position z-score (final layer)	0.993	0.998	0.952	0.831	16.2
	Layer 32, $1-\cos$	0.959	0.975	0.979	0.904	16.2
	Layer 32 + position z-score	0.994	0.999	0.966	0.871	16.2
	Layer 32 + Mahalanobis d=128[a]	0.991	0.998	0.983	0.928	16.2
	kNN reference, layer 32[b]	1.000	1.000	0.987	0.937	–
Capsule	Original baseline (final layer, $1-\cos$)	0.522	0.512	0.684	0.098	16.0
	+ position z-score (final layer)	0.622	0.672	0.901	0.508	16.0
	Layer 32, $1-\cos$	0.543	0.660	0.836	0.309	16.0
	Layer 32 + position z-score	0.721	0.767	0.931	0.666	16.0
	Layer 32 + Mahalanobis d=128[a]	0.600	0.647	0.956	0.720	16.0
	kNN reference, layer 32[b]	0.909	0.909	0.962	0.771	–
Hazelnut	Original baseline (final layer, $1-\cos$)	0.802	0.897	0.907	0.577	16.1
	+ position z-score (final layer)	0.864	0.939	0.969	0.812	16.1
	Layer 32, $1-\cos$	0.946	0.989	0.986	0.877	16.6
	Layer 32 + position z-score	0.981	0.993	0.989	0.901	16.6
	Layer 32 + Mahalanobis d=128[a]	0.960	0.969	0.987	0.894	16.6
	kNN reference, layer 32[b]	0.999	1.000	0.995	0.941	–
Mean	Original baseline (final layer, $1-\cos$)	0.750	0.787	0.840	0.480	16.1
	+ position z-score (final layer)	0.826	0.870	0.941	0.717	16.1
	Layer 32, $1-\cos$	0.816	0.875	0.934	0.697	16.3
	Layer 32 + position z-score	0.899	0.920	0.962	0.813	16.3
	Layer 32 + Mahalanobis d=128[a]	0.850	0.871	0.975	0.847	16.3
	kNN reference, layer 32[b]	0.969	0.970	0.981	0.883	–

Table 5.7: Paired-bootstrap change in image AUROC on the test split, 95 % interval. Asterisk: interval excludes zero. The first four rows use configurations fixed before their test evaluation; the last three evaluate the layer-32 re-selection.

Comparison (new vs reference)	Bottle	Capsule	Hazelnut
Final layer + z-score (top-3) vs original (max)	+0.073 [+0.019, +0.152]*	+0.150 [−0.045, +0.356]	+0.137 [+0.054, +0.226]*
Layer 32 + z-score (top-3) vs original (max)	+0.075 [+0.021, +0.153]*	+0.246 [+0.049, +0.457]*	+0.191 [+0.107, +0.284]*
Layer 32 + z-score vs final layer + z-score (top-3)	+0.002 [+0.000, +0.007]	+0.095 [+0.038, +0.150]*	+0.054 [+0.017, +0.101]*
Layer 32, $1-\cos$ vs original (max)	+0.034 [−0.024, +0.110]	+0.021 [−0.075, +0.114]	+0.144 [+0.061, +0.239]*
Layer 32 Mahalanobis vs layer 32 $1-\cos$ (max)	+0.033 [+0.004, +0.078]*	+0.057 [−0.123, +0.259]	+0.013 [−0.031, +0.068]
Layer 32 Mahalanobis (max) vs layer 32 z-score (top-3)	−0.008 [−0.029, +0.002]	−0.167 [−0.282, −0.057]*	−0.034 [−0.067, −0.007]*
Layer 32 Mahalanobis (max) vs original (max)	+0.067 [+0.012, +0.144]*	+0.079 [−0.134, +0.303]	+0.157 [+0.076, +0.254]*

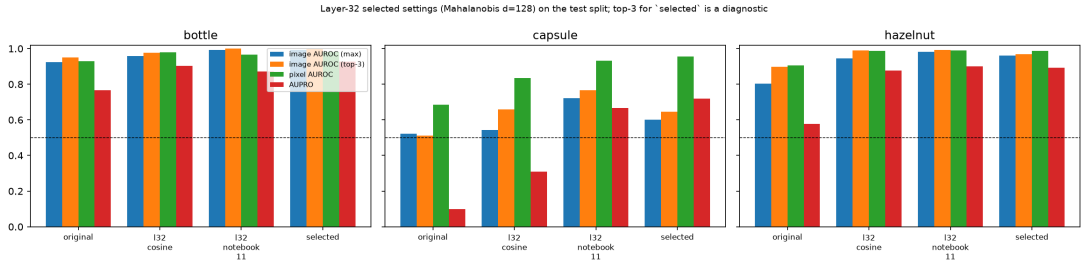


Figure 5.5: Layer-32 predictor with the selected Mahalanobis scoring on the test split, compared with the other rows of Table 5.6.

5.3.3 A negative result: re-selection for layer 32

Because the settings were chosen for the final-layer predictor, the same protocol was repeated on the layer-32 predictors and synthetic data, again without touching the test split. It chose a Mahalanobis residual score (PCA to 128 components, fitted on nominal validation residuals) with the max score, which was then scored once on the test split. Against raw cosine on the same predictor it improves bottle (+0.033, [+0.004, +0.078]) and is inconclusive for capsule and hazelnut, and its pixel AUROC and AUPRO are higher (means 0.976 and 0.847, point estimates). At image level, however, it is *worse* than the z-score configuration (capsule -0.167, $[-0.282, -0.057]$; hazelnut -0.034, $[-0.067, -0.007]$). The configuration fixed before the re-selection is therefore reported as the headline and the Mahalanobis result as a negative finding on how well synthetic validation transfers.

5.3.4 Pretrained predictor and context leakage

Our masking acts on latents after full-image encoding, so context tokens may already contain the defect. As a first test, Meta's pretrained I-JEPA context encoder and predictor were run on the visible tokens only (zero-shot, ImageNet-22K). It did not beat our predictor (image AUROC with the z-score: bottle 0.986 against 0.998, capsule 0.647 against 0.672, hazelnut 0.809 against 0.939) and costs about 980 ms per image against 23 ms. Leakage is therefore not supported as the main limit, but the test is confounded by predictor size and training data.

5.4 Discussion

The pipeline is decoder-free, fits in 8 GB and runs end to end in 134 ms. Latent prediction gives a usable localisation signal for categories with regular structure, and the coarse scheme obtains nearly the same localisation as dense masking with 8 % of the predictor passes. The adaptive fine pass did not add accuracy with the default gate and added only inconsistent gains with a calibrated gate, so its value in this pilot is limited to avoiding dense masking, which fixed masking also achieves.

The largest gains came not from the masking policy but from the input features and the score normalisation: mid-level tokens (layer 32) and a per-position z-score recover much of the gap to the nearest-neighbour reference and lift capsule from chance level to 0.77 image AUROC. A nearest-neighbour reference on the same layer-32 tokens remains stronger at image level (mean 0.97 against 0.92), so the gap to a memory-bank method is reduced, not closed. The re-selection experiment shows that a synthetic vali-

dation criterion close to its ceiling can prefer settings that do not transfer to image-level test performance.

5.5 Limitations

- **Scope.** Only 3 of 15 categories were evaluated; the other 12, MVTec AD 2 and a faithful PatchCore and MAE baseline are outstanding.
- **Sample size and seeds.** Validation sets have 21–39 images and test sets 20–40 normal images, and each predictor is a single training run per category, so differences of a few points are within noise.
- **Synthetic validation.** CutPaste and Perlin defects are a stand-in for real defects; no proxy-versus-test check exists for layer 32.
- **Possible context leakage.** Masking after full-image encoding was only tested zero-shot; retraining on input-masked latents was not done.
- **Reference baseline.** The nearest-neighbour row is not the original PatchCore.
- **Latency scope.** Cached-latent scoring excludes encoding and single-run latencies vary between sessions.
- **Validation reuse.** The same validation images are used for early stopping, calibration and selection, which makes false-positive control slightly optimistic.

CHAPTER

6

6. CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This project built and evaluated I-JEPA-AD, a decoder-free anomaly detector that predicts masked regions in the latent space of a frozen I-JEPA ViT-g/16 and refines suspicious regions with a gated fine pass. On a three-category MVTec AD pilot it localises defects with a mean pixel AUROC of 0.84 using 16 predictor passes per image and 134 ms end-to-end latency on an 8 GB GPU, a $2.6\times$ speed-up over dense masking at equal localisation quality. The adaptive gate did not give a consistent accuracy benefit, capsule defects were not detected by the original pipeline, and a nearest-neighbour baseline on the same latents was stronger.

Choosing the encoder layer and the score normalisation on synthetic defects, without the test split, then gave large gains: with the same 16 masks per image the mean pixel AUROC rose to 0.96, the mean AUPRO from 0.48 to 0.81 and the mean top-3 image AUROC to 0.92, with paired-bootstrap improvements over the original pipeline in every category and capsule rising from near chance to 0.77. A nearest-neighbour reference on the same tokens remains stronger at image level, and a layer-32 re-selection did not transfer to the test split at image level. Validation-only selection with bootstrap intervals makes these conclusions auditable.

6.2 Future Scope

- Evaluate the remaining twelve MVTec AD categories and MVTec AD 2 with multiple seeds and a validation-locked configuration.
- Train predictors on layers 24 and 28 and on layer concatenations, and study mask block size, augmentation and training length on the synthetic validation set.
- Retrain on input-masked context latents to measure context leakage decisively.
- Add a faithful PatchCore and MAE baseline under the same protocol.

- Investigate higher input resolution or an object crop for capsule, which changes the token grid and needs more memory and engineering.

BIBLIOGRAPHY

[1] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas, "Self-supervised learning from images with a joint-embedding predictive architecture," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023, pp. 15619–15629.

[2] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 16000–16009.

[3] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "MVTec AD — a comprehensive real-world dataset for unsupervised anomaly detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 9592–9600.

[4] P. Bergmann, K. Batzner, M. Fauser, D. Sattlegger, and C. Steger, "The MVTec anomaly detection dataset: A comprehensive real-world dataset for unsupervised anomaly detection," *Int. J. Comput. Vis.*, vol. 129, no. 4, pp. 1038–1059, 2021.

[5] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, and P. Gehler, "Towards total recall in industrial anomaly detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 14318–14328.

[6] T. Defard, A. Setkov, A. Loesch, and R. Audigier, "PaDiM: A patch distribution modeling framework for anomaly detection and localization," in *Int. Conf. Pattern Recognit. (ICPR) Workshops*, 2021, pp. 475–489.

[7] V. Zavrtanik, M. Kristan, and D. Skočaj, "DRAEM — a discriminatively trained reconstruction embedding for surface anomaly detection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2021, pp. 8330–8339.

[8] V. Zavrtanik, M. Kristan, and D. Skočaj, "Reconstruction by inpainting for visual anomaly detection," *Pattern Recognit.*, vol. 112, art. 107706, 2021.

[9] C.-L. Li, K. Sohn, J. Yoon, and T. Pfister, "CutPaste: Self-supervised learning for anomaly detection and localization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 9664–9674.

[10] K. Batzner, L. Heckler, and R. König, “EfficientAD: Accurate visual anomaly detection at millisecond-level latencies,” in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV)*, 2024, pp. 128–138.

[11] S. Akcay, D. Ameln, A. Vaidya, B. Lakshmanan, N. Ahuja, and U. Genc, “Anomalib: A deep learning library for anomaly detection,” in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, 2022, pp. 1706–1710.

[12] Y. LeCun, “A path towards autonomous machine intelligence, version 0.9.2,” OpenReview, 2022.

A. PUBLICATIONS AND INTELLECTUAL PROPERTY

This annexure records project-related publications and intellectual property.

Table A.1: Publications and Intellectual Property

Type	Title / Description	Status
Research Paper	I-JEPA-AD: Decoder-Free Latent Anomaly Detection with Coarse-to-Fine Adaptive Masking (IEEE conference format)	Manuscript prepared; submission status to be updated

B. STUDENT CONTRIBUTION STATEMENT

Each student should provide a specific and verifiable description of their contribution. The entries below are to be completed by the students.

Table B.1: Student Contribution Statement

Student Name	Roll Number	Contribution
Aryan Jawanjal	A-35	
Akshat Tiwari	A-11	
Arnav Milmile	A-30	
Aashutosh Dubey	A-02	

C. PROJECT OUTPUTS

Major outputs of the project:

- Software: the `ijepa_ad` package, training and evaluation scripts, the Streamlit dashboard and 376 automated tests.
- Trained predictors for bottle, capsule and hazelnut for both the final-layer and the layer-32 tokens.
- Notebooks 05 to 13 documenting every experiment, with saved result files, the synthetic validation set generator and the selection records.
- Project repository: `https://github.com/Auxilus08/I-JEPA-AD`.
- Research paper in IEEE conference format describing the method and results.